

TD/TP 6

Informatique 2 : Algorithmique 2 / Python MIP - S2

Exercice 1 :

1. Définir la complexité
2. Quelles sont les étapes à suivre pour calculer la complexité temporelle d'un algorithme
3. Que désigne $O(1)$
4. Quelle est la complexité qui a le temps d'exécution le plus lent

Exercice 2 : (toutes les fonctions ci-dessous sont déjà vues dans le cours)

Déterminer la complexité des programmes suivants (avec calcul de nombre d'opérations primitives)

```
def verifier(a, b):
    if a > b:
        return True
    else:
        return False
```

```
def fct(c):
    return c*2/5 + 4
```

```
def recherche_binaire(data, value):
    n = len(data)
    left = 0
    right = n - 1
    while left <= right:
        middle = (left + right) // 2
        if value < data[middle]:
            right = middle - 1
        elif value > data[middle]:
            left = middle + 1
        else:
            return middle
    return ValueError("Valeur n'existe pas dans la liste")

valeurs = [1, 2, 3, 4, 5, 6, 7, 8, 9]
print(recherche_binaire(valeurs, 5))
```

4

```
def recherche_lineaire(arr, cible):
    for i in range(len(arr)):
        if arr[i] == cible:
            return i
    return "cible non trouvée"

data = [1, 2, 9, 8, 3, 4, 7, 6, 5]
print(recherche_lineaire(data, 9))
```

2

```
def tri_bulles(lst):
    n = len(lst)
    for i in range(n):
        for j in range(0, n-i-1):
            if lst[j] > lst[j+1]:
                # Échanger les éléments si ils sont dans le mauvais ordre
                lst[j], lst[j+1] = lst[j+1], lst[j]
```

```

def tri_fusion(arr):
    if len(arr) > 1:
        milieu = len(arr) // 2
        moitie_gauche = arr[:milieu]
        moitie_droite = arr[milieu:]

        tri_fusion(moitie_gauche)    ## appel récursif
        tri_fusion(moitie_droite)    ## appel récursif

        i = j = k = 0

        while i < len(moitie_gauche) and j < len(moitie_droite):
            if moitie_gauche[i] < moitie_droite[j]:
                arr[k] = moitie_gauche[i]
                i += 1
            else:
                arr[k] = moitie_droite[j]
                j += 1
            k += 1

        while i < len(moitie_gauche):
            arr[k] = moitie_gauche[i]
            i += 1
            k += 1

        while j < len(moitie_droite):
            arr[k] = moitie_droite[j]
            j += 1
            k += 1

    return arr

```

```

def fibonacci(n):
    if n <= 1:
        return n
    else:
        return fibonacci(n-1) + fibonacci(n-2)

```

Exercice 3 :

Déterminer la complexité des programmes suivants (avec calcul de nombre d'opérations primitives)

```

def fct(n):
    for i in range(n):
        print(i)
        break

```

```

def puissance_inferieur_a_n(n):
    if n==1:
        return 0

    puissance = 1
    while 2 ** puissance <= n:
        puissance += 1
    return puissance-1 , 2**(puissance-1)

```

```
memo = {}

def fibonacci_recursive_memo(n):
    ## si le n à calculer est dans notre dictionnaire --> retourne le
    if n in memo:
        return memo[n]

    if n <= 1:
        return n ## résultat pour les indices 0 et 1

    ## sinon, calcule le, sauvegarde le dans memo, et retourne le
    else:
        memo[n] = fibonacci_recursive_memo(n-1) + fibonacci_recursive_memo(n-2)

    return memo[n]
```